



CLOUD COMPUTING: SERVICE MODELS

Module 2: Service Models & Deployment Strategies

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Outline

1 Infrastructure as a Service (IaaS)

2 PaaS, SaaS & Managed Services

3 Deployment Models

As A Service

-aaS

(DB)aaS

no management



Outline

 $O(7)$

1 Infrastructure as a Service (IaaS)

CC

2010

3010

1050

1180

index

 $O(\log 7)$

index	row
1050	2
1180	3
2010	0
3010	1

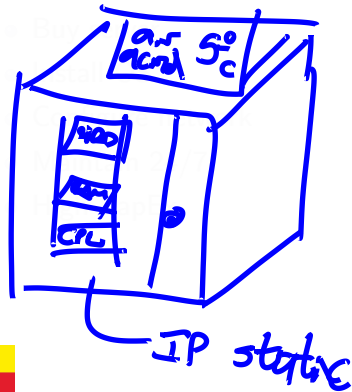


The Problem: Infrastructure without Hardware

Challenge

I need compute resources, but I cannot buy, install, or maintain physical servers. *How do I access infrastructure on demand?*

Traditional Approach



Cloud Approach

- Provision instantly
- Scale elastically
- Provider maintains
- Pay per use
- Low OpEx



The Problem: Infrastructure without Hardware

Challenge

I need compute resources, but I cannot **buy**, **install**, or **maintain** physical servers. *How do I access infrastructure on demand?*

Traditional Approach

- Buy server
- Install OS
- Configure network
- Maintain 24/7
- High CapEx

→ No GUY

Cloud Approach

- Provision instantly
- Scale elastically
- Provider maintains
- Pay per use
- Low OpEx



The Problem: Infrastructure without Hardware

Challenge

I need compute resources, but I cannot **buy**, **install**, or **maintain** physical servers. *How do I access infrastructure on demand?*

Traditional Approach

- Buy server
- Install OS
- Configure network
- Maintain 24/7
- High CapEx

Cloud Approach

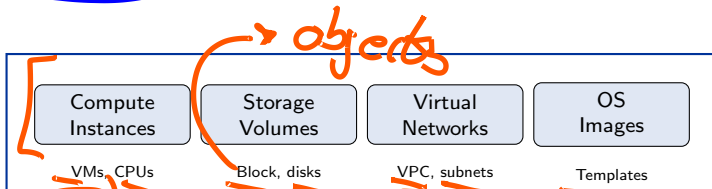
- Provision instantly
- Scale elastically
- Provider maintains
- Pay per use
- Low OpEx



What is IaaS?

Infrastructure as a Service

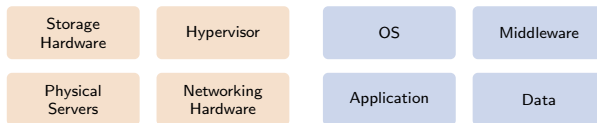
IaaS provides on-demand access to **compute**, **storage**, and **networking** resources. You control the infrastructure, the provider manages the physical hardware.



IaaS: Rent infrastructure, manage everything inside



Shared Responsibility: *Who Does What?*



Provider Responsibility

Provider Controls:

- Physical infrastructure
- Hypervisors
- Network connectivity
- Storage systems

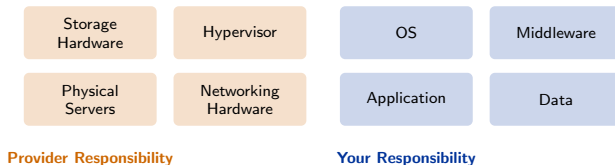
Your Responsibility

You Control:

- Operating systems
- Applications
- Data
- Security policies



Shared Responsibility: *Who Does What?*



Provider Controls:

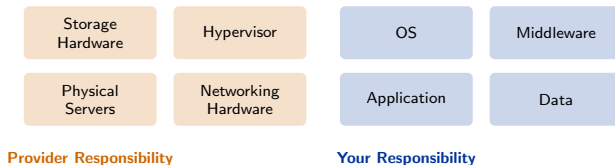
- Physical infrastructure
- Hypervisors
- Network connectivity
- Storage systems

You Control:

- Operating systems
- Applications
- Data
- Security policies



Shared Responsibility: *Who Does What?*



Provider Controls:

- Physical infrastructure
- Hypervisors
- Network connectivity
- Storage systems

You Control:

- Operating systems
- Applications
- Data
- Security policies



Key IaaS Advantages & Disadvantages

Advantages

- **Maximum control** over infrastructure
- **Flexible** — run any OS or software
- **Cost-efficient** — pay only for resources used
- **Scalable** — add/remove instances instantly
- **Global reach** — deploy in any region

Disadvantages

- **Operational burden** — you manage everything
- **Patching** — OS and security updates
- **Complexity** — networking, storage, security
- **Skills required** — infrastructure expertise
- **Vendor lock-in risk** — APIs are vendor-specific



Key IaaS Advantages & Disadvantages

Advantages

- **Maximum control** over infrastructure
- **Flexible** — run any OS or software
- **Cost-efficient** — pay only for resources used
- **Scalable** — add/remove instances instantly
- **Global reach** — deploy in any region

Disadvantages

- **Operational burden** — you manage everything
- **Patching** — OS and security updates
- **Complexity** — networking, storage, security
- **Skills required** — infrastructure expertise
- **Vendor lock-in risk** — APIs are vendor-specific

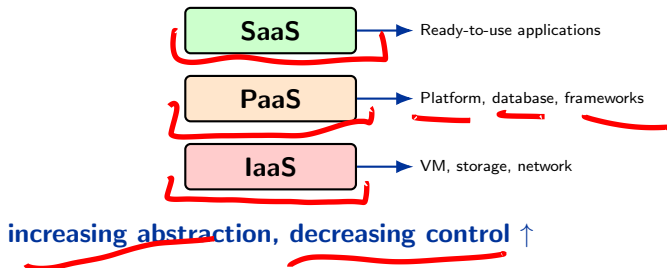


Outline

- 1 Infrastructure as a Service (IaaS)
- 2 PaaS, SaaS & Managed Services
- 3 Deployment Models



The *Abstraction Ladder*



Key Insight

As abstraction increases, you control less but manage less. Choose based on your team's expertise and business needs.



PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- Deployment platform: *code* → auto-deployed to production
- Managed database: PostgreSQL, MongoDB as a service
- Infrastructure as a Service: AWS, Azure, GCP
- Monitoring, built-in observability and logging

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- Deployment platform: code → auto-deployed to production

- Managed database: PostgreSQL, MongoDB, Redis, service

- Middleware: frameworks, caching, load balancing

- Automatic scaling: adjust capacity based on demand

- Monitoring: built-in observability and logging

GitHub Actions

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- **Deployment platform:** *code* → auto-deployed to production
- **Managed database:** PostgreSQL, MongoDB as a service
- **Middleware:** *frameworks, caching, load balancing*
- **Automatic scaling:** *adjust capacity based on demand*
- **Monitoring:** *built-in observability and logging*

Supabase *Atlas*

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- **Deployment platform**: *code* → auto-deployed to production
- **Managed database**: PostgreSQL, MongoDB as a service
- **Middleware**: *frameworks*, *caching*, *load balancing*
- Automatic scaling: adjust capacity *based on demand*
- Monitoring: built-in *observability* and *logging*

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- **Deployment platform**: *code* → auto-deployed to production
- **Managed database**: PostgreSQL, MongoDB as a service
- **Middleware**: *frameworks, caching, load balancing*
- **Automatic scaling**: adjust capacity *based on demand*
- **Monitoring**: built-in *observability and logging*

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

server ↓ 10 replicas

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- **Deployment platform**: *code* → auto-deployed to production
- **Managed database**: PostgreSQL, MongoDB as a service
- **Middleware**: *frameworks, caching, load balancing*
- **Automatic scaling**: adjust capacity *based on demand*
- **Monitoring**: built-in observability and logging *error*

Example

Deploy a Node.js app: *push code* → platform builds, tests, scales automatically.

PaaS: Platform as a Service

Definition

PaaS provides a **platform** where you deploy applications *without managing infrastructure*. The provider handles OS, databases, and middleware.

Typical PaaS Services:

- **Deployment platform**: *code* → auto-deployed to production
- **Managed database**: PostgreSQL, MongoDB as a service
- **Middleware**: *frameworks, caching, load balancing*
- **Automatic scaling**: adjust capacity *based on demand*
- **Monitoring**: built-in *observability* and *logging*

Example

Deploy a Node.js app: push code → platform builds, tests, scales automatically.

Gith

data unit test

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- Productivity: Gmail, Office 365, Google Workspace
- CRM: Salesforce, HubSpot
- ERP: SAP S/4HANA, Microsoft Teams
- Analytics: Tableau, Power BI
- Business Intelligence: Data warehouse as a Service

Key Characteristic

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- Productivity: Gmail, Office 365, Google Workspace
- CRM: Salesforce, HubSpot
- Collaboration: Slack, Microsoft Teams
- Analytics: Tableau, Looker
- Business intelligence: Data warehouses as SaaS

Key Characteristic

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- **Productivity:** Gmail, Office 365, Google Workspace
- **CRM:** Salesforce, HubSpot
- **Collaboration:** Slack, Microsoft Teams
- **Analytics:** Custom
- **Business intelligence:** Data warehouses as SaaS

Key Characteristic

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- **Productivity:** Gmail, Office 365, Google Workspace
- **CRM:** Salesforce, HubSpot
- **Collaboration:** Slack, Microsoft Teams
- **Analytics:** Tableau, Looker
- **Business intelligence:** Data warehouses as SaaS

Key Characteristic

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- **Productivity:** Gmail, Office 365, Google Workspace
- **CRM:** Salesforce, HubSpot
- **Collaboration:** Slack, Microsoft Teams
- **Analytics:** Tableau, Looker
- **Business Intelligence:** Data warehouses as SaaS

Key Characteristics

Multi-tenant: multiple customers share the same infrastructure. Your data is isolated but the platform is shared.

SaaS: Software as a Service

Definition

SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- **Productivity**: Gmail, Office 365, Google Workspace
- **CRM**: Salesforce, HubSpot
- **Collaboration**: Slack, Microsoft Teams
- **Analytics**: Tableau, Looker
- **Business intelligence**: Data warehouses as SaaS

Key Characteristics

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.

Handwritten notes:
Foldg
AWS Redshift

SaaS: Software as a Service

Definition

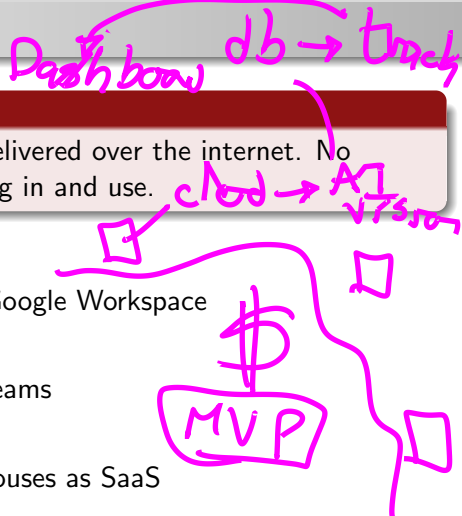
SaaS is a **ready-to-use application** delivered over the internet. No installation, no configuration — just log in and use.

Typical SaaS Examples:

- **Productivity**: Gmail, Office 365, Google Workspace
- **CRM**: Salesforce, HubSpot
- **Collaboration**: Slack, Microsoft Teams
- **Analytics**: Tableau, Looker
- **Business intelligence**: Data warehouses as SaaS

Key Characteristic

Multi-tenant: thousands of customers share the same infrastructure. Your data is isolated, but the platform is shared.



IaaS vs. PaaS vs. SaaS: Comparison

membership = Oper + cloud comp + revenue

Aspect	IaaS	PaaS	SaaS
You manage	OS, App, Data	App, Data	Nothing
Provider manages	Hardware	Infrastructure	Everything
Control level	High	Medium	Low
Flexibility	Very high	Medium	Limited
Effort	High	Low	Minimal
Cost model	Pay per VM	Pay per platform	Pay per user
Vendor lock-in	Medium	High	Highest
Time to deploy	Hours	Minutes	Instant

DO IoT → DO Database → DO Data warehouse

Erz (Amazon Docker) → DO Focus

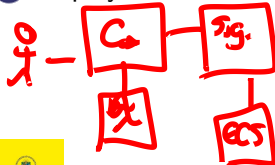


Outline

1 Infrastructure as a Service (IaaS)

2 PaaS, SaaS & Managed Services

3 Deployment Models



FBX
U30X
TwilioX
JIRA X



detect zp

block

DDoS

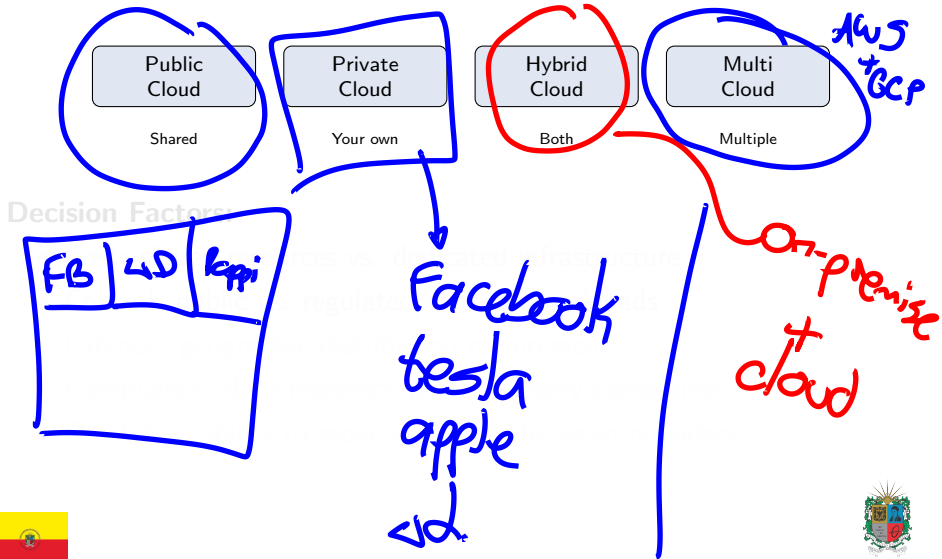
fake IP

AWS Cognito

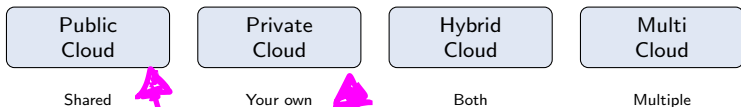
↓
rasonware



Deployment Models: *Where Lives Your Infrastructure?*



Deployment Models: *Where Lives Your Infrastructure?*

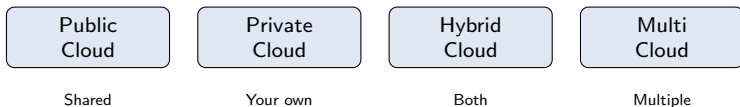


Decision Factors:

- **Cost:** shared resources vs. dedicated infrastructure
- Control: public vs. regulated/sensitive workloads
- Latency: geographic distribution requirements
- Compliance: data residency and regulatory constraints
- Flexibility: ability to move workloads between providers



Deployment Models: *Where Lives Your Infrastructure?*

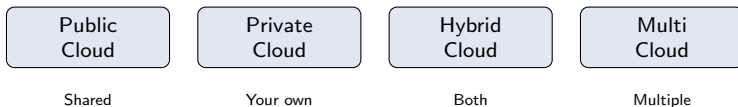


Decision Factors:

- **Cost:** shared resources vs. dedicated infrastructure
- **Control:** public vs. regulated/sensitive workloads
- Latency: geographic distribution requirements
- Compliance: data residency and regulatory constraints
- Flexibility: ability to move workloads between providers



Deployment Models: *Where Lives Your Infrastructure?*

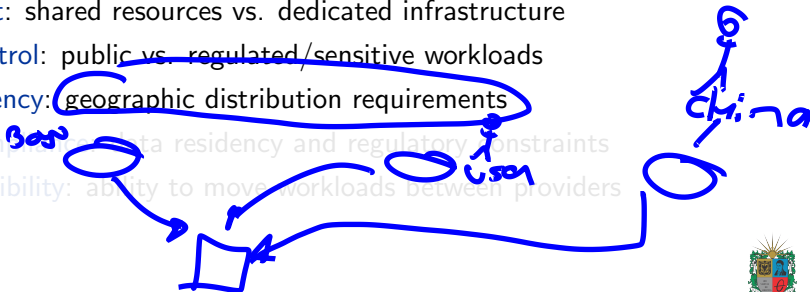


Decision Factors:

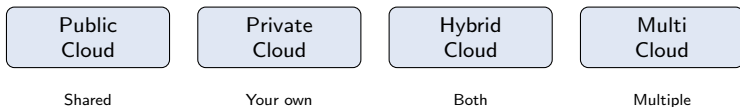
- **Cost:** shared resources vs. dedicated infrastructure
- **Control:** public vs. regulated/sensitive workloads
- **Latency:** geographic distribution requirements

• Compliance: data residency and regulatory constraints

• Flexibility: ability to move workloads between providers



Deployment Models: *Where Lives Your Infrastructure?*

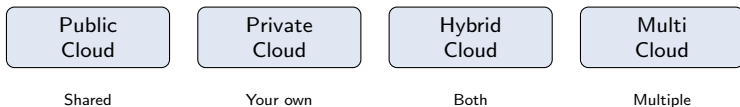


Decision Factors:

- **Cost:** shared resources vs. dedicated infrastructure
- **Control:** public vs. regulated/sensitive workloads
- **Latency:** geographic distribution requirements
- **Compliance:** data residency and regulatory constraints
- **Flexibility:** ability to move workloads between providers



Deployment Models: *Where Lives Your Infrastructure?*



Decision Factors:

- **Cost:** shared resources vs. dedicated infrastructure
- **Control:** public vs. regulated/sensitive workloads
- **Latency:** geographic distribution requirements
- **Compliance:** data residency and regulatory constraints
- **Flexibility:** ability to move workloads between providers



Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. Accessed over the internet.

Characteristics:

1000 CPUs - 1 Tb L4M

client

2 CPUs
56GB RAM

16GB
900 CPUs
100GB RAM

Virt

Available
98 CPUs
405 GB RAM

GCP
Google
Cloud
Platform

Use Cases

Startups, web applications, batch processing, development/testing environments

Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- Lowest cost — shared infrastructure economy
- Maximum elasticity — scale instantly
- Global reach — data centers worldwide
- Less control — multi-tenant environment
- Compliance challenges — shared security responsibility

Provider

Use Cases

Startups, web applications, batch processing, development/testing environments.

Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- Lowest cost — shared infrastructure economy
- Maximum elasticity — scale instantly 
- Global reach — data centers worldwide 
- Less control — multi-tenant environment
- Compliance challenges — shared security responsibility

Use Cases

Startups, web applications, batch processing, development/testing environments.

Public Cloud

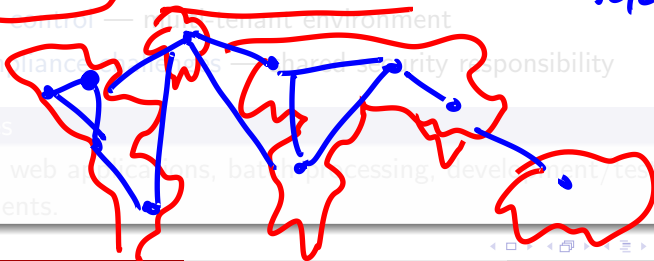
Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- Lowest cost — shared infrastructure economy
- Maximum elasticity — scale instantly
- Global reach — data centers worldwide

low latency



Use Cases

Startups, web applications, batch processing, development/testing environments.

Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- Lowest cost — shared infrastructure economy
- Maximum elasticity — scale instantly
- Global reach — data centers worldwide
- Less control — multi-tenant environment
- Compliance challenges — shared security responsibility

Use Cases

Startups, web applications, batch processing, development/testing environments.

Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- Lowest cost — shared infrastructure economy
- Maximum elasticity — scale instantly
- Global reach — data centers worldwide
- Less control — multi-tenant environment
- Compliance challenges — shared security responsibility

Use Cases

Startups, web applications, batch processing, development/testing environments.

Public Cloud

Definition

Infrastructure is **shared** by multiple customers and **managed** by a cloud provider. *Accessed over the internet.*

Characteristics:

- **Lowest cost** — shared infrastructure economy
- **Maximum elasticity** — scale instantly
- **Global reach** — data centers worldwide
- **Less control** — multi-tenant environment
- **Compliance challenges** — shared security responsibility

Digital Ocean
 GCP
 AWS
 Azure
 Oracle
 IBM/whatsapp

Use Cases

Startups, web applications, batch processing, development/testing environments.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control over dedicated resources
- Enhanced security and privacy concerns

dedicated resources (1-3 years)

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — (dedicated resources)
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — dedicated resources
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — dedicated resources
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — dedicated resources
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — dedicated resources
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

vertical

Private Cloud

Definition

Infrastructure is **dedicated** to your organization. May be hosted on-premises or by a provider, but is **exclusively yours**.

Characteristics:

- Maximum control — dedicated resources
- Enhanced security — no multi-tenant concerns
- Compliance-friendly — data residency control
- Higher cost — no economy of scale
- Limited elasticity — depends on physical capacity

Use Cases

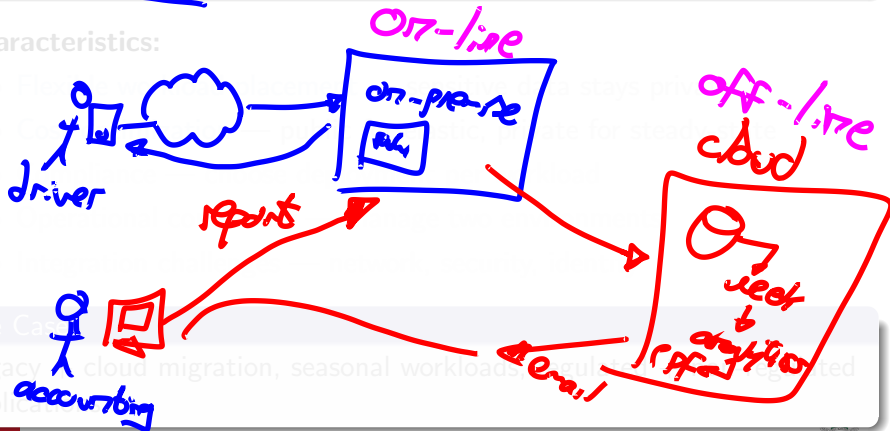
Banks, healthcare systems, regulated industries, organizations with strict data sovereignty requirements.

Hybrid Cloud

Definition

A **combination** of *public* and *private* cloud. Workloads move between both based on requirements, costs, and compliance.

Characteristics:



Hybrid Cloud

Definition

A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- **Flexible workload placement** — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- Integration challenges — network, security, identity

Use Cases

Legacy + cloud migration, seasonal workloads, regulated + non-regulated applications.

Hybrid Cloud

Definition

A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- Flexible workload placement — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- Integration challenges — network, security, identity

Use Cases

Legacy + cloud migration, seasonal workloads, regulated + non-regulated applications.

Hybrid Cloud

Definition

A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- Flexible workload placement — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- legal challenges — network, security, identity

Use Cases
Policies

Legacy + cloud migration, seasonal workloads, regulated + non-regulated applications.

Hybrid Cloud

Definition

A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- Flexible workload placement — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- Integration challenges — network, security, identity

Use Cases

Legacy + cloud migration, sensitive workloads, regulated + non-regulated applications.

Costs

Red Hat



not the same

Assure



Hybrid Cloud

Definition

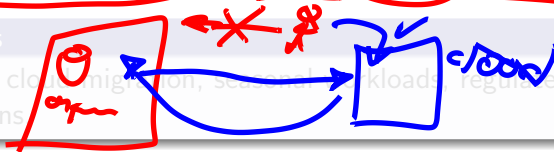
A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- Flexible workload placement — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- Integration challenges — network, security, identity

Use Cases

Legacy + cloud migration, seasonal workloads, regulated + non-regulated applications



Hybrid Cloud

Definition

A **combination** of *public and private cloud*. Workloads move between both based on requirements, costs, and compliance.

Characteristics:

- Flexible workload placement — sensitive data stays private
- Cost optimization — public for elastic, private for steady-state
- Compliance — choose deployment per workload
- Operational complexity — manage two environments
- Integration challenges — network, security, identity

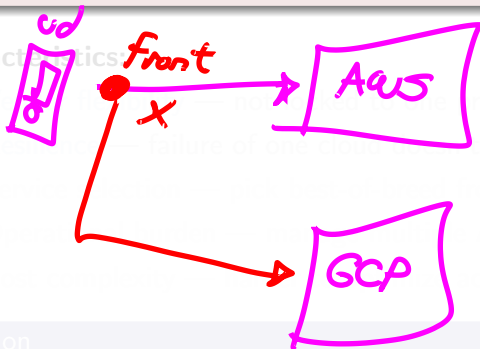
Use Cases

Legacy + cloud migration, seasonal workloads, regulated + non-regulated applications.

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.



Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics.

- **Vendor flexibility** — not locked to one provider *AWS*
- Resilience — failure of one cloud doesn't stop everything
- Service selection — pick best-of-breed from each provider
- Operational burden — manage multiple APIs, *GCP*
- Cost complexity — harder to optimize across providers

Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics:

- **Vendor flexibility** — not locked to one provider
 - **Resilience** — failure of one cloud doesn't stop everything
 - Service selection — pick best-of-breed from each provider
 - Operational burden — manage multiple APIs, toolchains
 - Cost complexity — harder to optimize across providers
- Handwritten red annotations: "second cloud" with an arrow pointing from the word "second" to the word "cloud" in the list item "Service selection".*

Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics:

- **Vendor flexibility** — not locked to one provider
- **Resilience** — failure of one cloud doesn't stop everything
- **Service selection** — pick best-of-breed from each provider
- Operational burden — manage multiple APIs, toolchains
- Cost complexity — harder to optimize across providers

Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics:

- **Vendor flexibility** — not locked to one provider
- **Resilience** — failure of one cloud doesn't stop everything
- **Service selection** — pick best-of-breed from each provider
- **Operational burden** — manage multiple APIs, toolchains
- **Cost complexity** — harder to optimize across providers

Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?



Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics:

- **Vendor flexibility** — not locked to one provider
- **Resilience** — failure of one cloud doesn't stop everything
- **Service selection** — pick best-of-breed from each provider
- **Operational burden** — manage multiple APIs, toolchains
- **Cost complexity** — harder to optimize across providers

Question

Multicloud solves technical problems, but adds operational complexity. Is it always worth it?

Multicloud

Definition

Multiple public cloud providers for the same application or organization.
Reduces vendor lock-in and improves resilience.

Characteristics:

- **Vendor flexibility** — not locked to one provider
- **Resilience** — failure of one cloud doesn't stop everything
- **Service selection** — pick best-of-breed from each provider
- **Operational burden** — manage multiple APIs, toolchains
- **Cost complexity** — harder to optimize across providers

Question

Multicloud solves **technical** problems, but adds **operational** complexity. Is it always worth it?

How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	2	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
		Total:	69/100

Steps:

1. List all requirements: functional and non-functional
2. Identify constraints: budget, compliance, skills
3. Score each criterion against criteria (1-5)
4. Weight by importance to your organization
5. Determine total score for each option and rank



How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	3	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
		Total:	69/100

Steps:

- 1 List all requirements: functional and non-functional
- 2 Identify constraints: budget, compliance, skills
- 3 Score each option against criteria (1–5)
- 4 Weight by importance to your organization
- 5 Document your decision in an Architecture Decision Record (ADR)



How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	3	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
		Total:	69/100

Steps:

- 1 List all **requirements**: functional and non-functional
- 2 Identify **constraints**: budget, compliance skills
- 3 Score each option against criteria (1-5)
- 4 Weight by importance to your organization
- 5 Document your decision in an Architecture Decision Record (ADR)



How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	3	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
		Total:	69/100

Steps:

- 1 List all **requirements**: functional and non-functional
- 2 Identify **constraints**: budget, compliance, skills
- 3 Score each option against criteria (1–5)
- 4 Weight by importance to your organization
- 5 Document your decision in an Architecture Decision Record (ADR)

rubric



How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	3	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
Total:			69/100

Steps:

- 1 List all **requirements**: functional and non-functional
- 2 Identify **constraints**: budget, compliance, skills
- 3 Score each option against criteria (1–5)
- 4 Weight by importance to your organization
- 5 Document your decision in an Architecture Decision Record (ADR)



How to Choose: Decision Matrix

Criterion	Weight	Score	Total
Cost	3	4	12
Security	5	3	15
Scalability	4	5	20
Compliance	5	2	10
Team skills	3	4	12
Total:			69/100

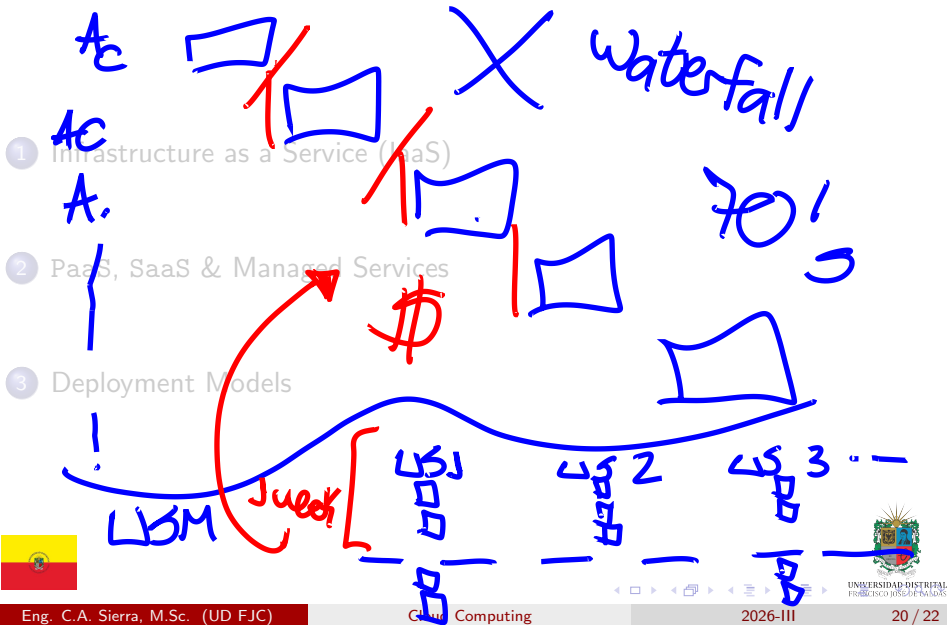
Platform Engineer

Steps:

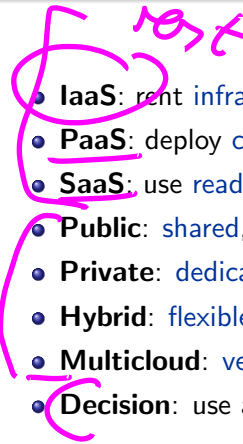
- 1 List all **requirements**: functional and non-functional
- 2 Identify **constraints**: budget, compliance, skills
- 3 Score each option against criteria (1–5)
- 4 Weight by importance to your organization
- 5 **Document** your decision in an Architecture Decision Record (ADR)



Outline



Recap: Module 2

- 
- **IaaS**: rent **infrastructure**, manage applications
 - **PaaS**: deploy **code**, platform handles infrastructure
 - **SaaS**: use **ready-made applications**
 - **Public**: **shared**, cost-efficient, least control
 - **Private**: **dedicated**, compliant, expensive
 - **Hybrid**: **flexible**, complex to operate
 - **Multicloud**: **vendor-independent**, hard to maintain
 - **Decision**: use a **matrix** to evaluate options objectively



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

